Designed and engineered, artfully — silicon to software.

BSc Computer Science, Manchester (2023–2026) · Incoming MSc Computer & Embedded Systems Engineering, TU Delft (Sept 2026)

INDEX / ALL PROJECTS

01	Quant Trading System (Automaton-QTS)	FINANCE
02	Cellular-Automaton SoC Accelerator	FINANCE
03	Aero-Lab — Quadrotor Flight Simulator	DRONE
04	Interpretable MPCC Racing Controller	MOTORSPORT
05	Stump CPU — Spartan-6 RISC Processor	FINANCE
06	Multicore Computing Labs	FINANCE
07	ONI — Rust LLM Agent Harness	FINANCE
08	DAWgit — Git for DAW Projects	DRONE
09	FSAI Driverless Racing Simulator	MOTORSPORT
10	CommonRoad C++ Port	MOTORSPORT
11	Slit Light Interference Simulator	DRONE
12	CarrLane — Engineering Part Chatbot	FINANCE

FIELD / FINANCE / QUANT

01 / SYSTEMS · CONTROL

Quant Trading System (Automaton-QTS)

Multi-signal trading engine with a tick-level, queue-position-aware HFT backtester.

Python | NautilusTrader | hftbacktest | PyTorch | TimescaleDB

CODE	COVERAGE	SIM ORDER LATENCY
40.9k LOC	81.9%	50 µs

ROLE

Sole author. Built the signal pipeline, risk engine (hard circuit breakers), dual backtester, and the regime-gated propagation-graph research module.

REPO / https://github.com/RandevRanjit/Automaton-QTS

02 / HARDWARE

Cellular-Automaton SoC Accelerator

A self-designed programmable Wolfram-rule accelerator on a RISC-V SoC — synthesised, implemented, demoed live on FPGA.

SystemVerilog | Vivado | Artix-7 | RISC-V

ACCELERATOR	WHOLE SOC	RULES
4,433 LUT / 1,467 FF	43.81% LUT · 96% BRAM	256 runtime-selectable

ROLE

Designed the unit_automaton accelerator (rule-as-lookup, 32-bit LFSR seed, compute/write FSM, byte-enabled framestore writes), integrated it into the drawing engine, drove it from a RISC-V MMIO program, and took it through Vivado synthesis + implementation.

PRIVATE · CASE STUDY (NO CODE LINK)

05 / HARDWARE

Stump CPU — Spartan-6 RISC Processor

Complete 16-bit multi-cycle RISC CPU hand-built in Verilog and closed on real Spartan-6 silicon at 98% slice occupancy.

Verilog | Xilinx ISE | Spartan-6

SLICE LUTS	OCCUPIED SLICES	TIMING SCORE
2,068 / 2,400 (86%)	591 / 600 (98%)	0 — all constraints met

ROLE

Designed the datapath (ALU, shifter, sign-extender, 8×16-bit register file, muxes), the 3-state control FSM (FETCH/EXECUTE/MEMORY), and the instruction decoder. Wrote Battleship in Stump assembly against a memory-mapped 8×8 LED matrix.

PRIVATE · CASE STUDY (NO CODE LINK)

---

06 / SYSTEMS

Multicore Computing Labs

Measured parallelism on 72 cores — NUMA-aware pthreads, fine-grained spinlock dining philosophers, and a temporal-blocking stencil.

C | C++ | pthreads | OpenMP

VECADD SPEEDUP	FINE-SPINLOCK THROUGHPUT	SERIAL FRACTION
30.7× on 72 cores	~1,039K meals/s @ 64T	~3% (bandwidth-bound)

ROLE

Sole author (three labs). Implemented NUMA first-touch affinity vecadd, fine-grained spinlock dining philosophers with lock-ordering, and a temporal-blocking std::barrier stencil. Benchmarked on mcore72 (72-core, 2-socket NUMA machine).

PRIVATE · CASE STUDY (NO CODE LINK)

---

07 / SYSTEMS

ONI — Rust LLM Agent Harness

78k LOC single-crate Rust research harness — parser/exec separation, graph working-memory, criterion µs/ns benchmarks, process-group SIGKILL sandbox.

Rust | tokio | criterion

CODEBASE	PARSER TARGET	DISPATCH TARGET	LIT DETECTOR TARGET
78,176 LOC Rust	**< 100 µs / 15-sample corpus**	**< 5 ms / dispatch**	**< 50 ns / 1 KB chunk**

ROLE

Sole author. Built the parser/exec separation, graph working-memory (UUIDv4 nodes, BFS gather, compaction), three Criterion bench targets, and the process-group SIGKILL sandboxing. Production-incident-driven — the kill pattern came from a real git-clone hang in a remote-benchmark run.

REPO / https://github.com/RandevRanjit/ONI

12 / SYSTEMS

CarrLane — Engineering Part Chatbot

LLM + function-calling retrieval pipeline over an engineering parts catalog, built during an AI/Software Engineering internship.

Python | LLM | vector search | Node

CATALOG	LLM	DATES
10,000+ parts	**GPT-4o-mini + function-calling**	**Jun–Aug 2025, Chennai**

ROLE

AI / Software Engineering Intern. Built the LLM + function-calling retrieval pipeline over the engineering parts catalog; improved indexing and data quality for natural-language part selection.

REPO / https://github.com/RandevRanjit/Carrlane-Chatbot

FIELD / DRONE / MUSIC

03 / CONTROL

Aero-Lab — Quadrotor Flight Simulator

From-scratch RK4 rigid-body dynamics with a cascaded PID + differential-flatness controller.

Python | NumPy | Gymnasium | SciPy

CODE	TESTS	CONTROLLER
~15.9k LOC	**322 functions**	**PID + diff-flatness**

ROLE

Sole author. Hand-rolled the 6-DOF physics (RK4, motor lag, gyroscopic torque, aero drag), the cascaded PID + differential-flatness feedforward controller, and the mixer pseudo-inverse.

REPO / https://github.com/RandevRanjit/Aero-Lab

---

08 / SYSTEMS

DAWgit — Git for DAW Projects

A real macOS daemon — semantic-delta version control for Ableton sessions, with content-addressed audio LFS and Unix-socket IPC.

Rust | tokio | libgit2 | roxmltree

ABLETON PARSER	AUDIO LFS	IPC
gzip-XML zero-copy parse (roxmltree)	**SHA-256 content-addressed store**	**Unix domain socket, newline-delimited JSON**

ROLE

Sole author. Built the daemon (FS watcher, IPC, async/sync boundary), the Ableton gzip-XML parser (1,306 LOC, zero-copy), the git object layer (branch-per-file commits without touching HEAD, three-way revert via merge_trees), and the SHA-256 content-addressed audio LFS.

PRIVATE REPOSITORY

---

11 / GRAPHICS

Slit Light Interference Simulator

Physically-correct single-slit Fraunhofer diffraction computed per-fragment on the GPU — sinc² intensity in a custom GLSL shader with live parameters.

JavaScript | Three.js | GLSL

CUSTOM SHADERS	DIFFRACTION MODEL
6 GLSL shaders	**physically-correct sinc² per-fragment**

ROLE

Sole author. Implemented the interference fragment shader (sinc² intensity formula, correct physical parametrisation), the live control UI, and the Three.js render loop.

REPO / https://github.com/RandevRanjit/Slit-Light-Interference-Sim

FIELD / MOTORSPORT

04 / CONTROL

Interpretable MPCC Racing Controller

Final-year dissertation — model-predictive contouring control on a Fiala-tire bicycle plant.

Python | CasADi | IPOPT | SciPy

HORIZON	SOLVER STEP	INTEGRATOR
N=20 @ dt=0.02s	**19.77 ms mean**	**Radau IIA (implicit)**

ROLE

Sole author (dissertation). Built the MPCC OCP, dual warm-starting, JIT compilation, the Fiala-tire plant, and the comparison harness against RRHC and DDP.

PRIVATE REPOSITORY

09 / SYSTEMS · CONTROL

FSAI Driverless Racing Simulator

34k LOC C++ fixed-timestep sim — RK4 with adaptive sub-stepping, closed perception loop (ONNX + SIFT + Kalman), nanosecond budget timers.

C++17 | CMake | ONNXRuntime | OpenCV | Eigen

CODEBASE	DYNAMICS	PERCEPTION LOOP
34,148 LOC C++	**RK4 + adaptive sub-stepping**	**ONNX + SIFT + Kalman map fusion**

ROLE

Sole author. Built the RK4 vehicle dynamics with adaptive sub-stepping, the three runtime-switchable vehicle models (MB/ST/STD), the closed perception pipeline (stereo FBO → async PBO readback → SIFT disparity → ONNX cone detector → Kalman tracker → BayesianMapper), and the ns budget timer infra.

REPO / https://github.com/RandevRanjit/FSAI_Simulation_C

---

10 / SYSTEMS · CONTROL

CommonRoad C++ Port

~11.6k LOC C++17 port of CommonRoad vehicle models (KS/ST/MB) as a clean static library with SimulationDaemon API and TypeScript web-sdk.

C++17/20 | CMake | Eigen | TypeScript

CODEBASE	VEHICLE MODELS	ST KINEMATIC FALLBACK
~11.6k LOC C++17	**KS / ST / MB**	**at |vx| < 0.1 m/s**

ROLE

Sole author. Ported the CommonRoad Python vehicle models (KS, ST, MB) to C++17, built the SimulationDaemon API with ModelTiming sub-step scheduler, telemetry layer, and a TypeScript web-sdk.

REPO / https://github.com/RandevRanjit/CommonRoadCXXPort